

AI Model Repository Suite:

Final Technical Report

Team: The Capstone Crew

Members:

Kyle Schwartz – Computer Information Systems – AI learning track

Joe Samples – Computer Information Systems – Cybersecurity learning track

Matthew Clippard – Computer Information Systems – Cybersecurity learning track

Luyanda Chitofu – Computer Information Systems – Business Administration learning track

Course: CS499-FX1: Senior Capstone

Stakeholder: Nipesh Pant and The SEMO Harrison College of Business and Computing

Instructor: Dr. Reshmi Mitra

Semester: Spring 2026

Date: April 28, 2026

Executive Summary

Problem Statement: Modern AI development often suffers from a "fragmentation gap" where model code is difficult to share, host, and execute in a unified, secure environment. Developers lack a streamlined, modular platform that manages the complexities of dependency installation, secure execution, and dynamic interface generation for diverse AI services. Current industry solutions like Amazon Lambda address the need for serverless execution but require extensive knowledge of cloud orchestration and IAM security. Conversely, platforms like Hugging Face Spaces simplify model hosting but necessitate manual frontend development, requiring some knowledge in interface code such as Gradio or Streamlit.

Proposed Solution: The Capstone Crew developed the **ModelHubAI**, a web-based, micro-services platform designed for the secure hosting and automated management of AI models. The system acts as a central hub that bridges the gap between model developers and end-users through a "single-point connection" architecture. The developer uploads their Python code as a .zip or Python file, inputs information about how they want the interface to appear, and the system automatically serves the interface and runs the service. The developer only needs to create their AI model. The platform handles the deployment to end users.

Key Components

- **Django Backend:** Manages user authentication, the model catalog, and persistent data storage.
- **PostgreSQL Database:** Stores user data, model metadata, and execution metrics.
- **FastAPI AI Suite:** Handles model preparation, dependency parsing, and manages sibling containers for execution.
- **Secure Sandbox:** An isolated Docker-based Python runtime environment for running untrusted user code.

Major Results and Testing Highlights The platform successfully demonstrated the ability to automatically prepare and run model files across six different model types.

- **Performance:** Stress testing showed successful concurrent execution of three models with a minimal performance degradation of less than 10 seconds.
- **Security:** The system achieved network isolation, prevented Docker socket exploitation, and enforced resource limits when running untrusted user code.
- **User Experience:** A user interface is automatically generated by the system for the users based on input by the developer uploading the code.

Overall Impact This project provides a scalable blueprint for AI platforms that reduce the technical overhead of AI model deployment. This platform tailors to small developers and innovators that may have developed useful AI models, but don't have the resources or

knowledge to deploy them to end-users. The system uses a zero-trust execution policy that isolates untrusted code to ensure that user data and the system architecture remain secure.

Biographical Background



Kyle Schwartz (Lead Developer): Pursuing a degree in Computer Information Systems with an AI learning track. Kyle's best academic experiences at SEMO involved numerous software engineering projects where he gained experience in the software development lifecycle. Notable projects he has worked on include a SaltStack script deployment system for the SEMO Cyber Defense team where he integrated secure proxy servers and RESTful API architectures with front-end applications, an AWS-based e-commerce website where he engineered a cloud-based Python Django project, and an HTML based personal portfolio website where he mastered web

technologies and front-end design. During his time at SEMO, Kyle has also worked on smaller projects that sharpened his skills in machine learning, database development and management, data analysis, user experience design and testing, and mobile application development. His technical strengths include Python, Java, SQL, full-stack web development, and system architecture design and development. Kyle was the technical lead of this project, overseeing the architecture, full-stack development, and testing phases. He was responsible for designing and building the core architecture of the project, mainly focusing on the AI Service aspect of the system that required gaining knowledge in Docker-based microservices and dynamic dependency resolution to enable automated, zero-trust code deployment.



Joe Samples (Security Lead): Pursuing a degree in Computer Information Systems with a Cybersecurity learning track. Joe serves as the project's security lead by bringing a specialized focus to Cybersecurity within his Computer Information Systems major. During his academic career, he has developed a particular interest in web design and project management techniques. This dual interest allows him to approach development with both a creative eye and a structured, security-first mindset. His primary strengths include writing clean, efficient HTML code for the website's front-end architecture and Python-based security scripting.

Additionally, he is a proficient technical writer, and he helps ensure that complex project reports and documents remain clear and professional. Joe played a key role in the project's foundational stages, with his skill set coming in handy when the group was developing the Software Requirements Specification document. His contributions were equally vital during the implementation phase, where he was responsible for building the website's core

pages. By writing the front-end HTML and successfully integrating it with the project's backend infrastructure, Joe ensured a seamless and secure user experience.



Matthew Clippard (Integration Specialist): Pursuing a degree in Computer Information Systems with a Cybersecurity learning track. Matthew's best academic experiences at Southeast Missouri State University involved working on projects that require strong backend development, database management, and system integration. Throughout his coursework, he has gained hands on experience designing and implementing systems that rely on efficient data handling and communication between multiple services. These experiences helped him develop a strong understanding of how different components of a system interact and how to ensure reliable

performance across those components. During his time at SEMO, Matthew has worked on projects that strengthened his skills in Python programming, PostgreSQL database design, and API integration. He has also gained experience in web application security, including vulnerability awareness, input validation, and data sanitization to help protect systems against common attacks. His technical strengths include database optimization and building scalable connections between services. In this capstone project, Matthew served as the Integration Specialist, where he focused on managing communication between the Fast API AI service and the PostgreSQL Backend. He was responsible for ensuring that execution data, performance metrics, and model information were consistently stored and accurately retrieved. His contributions helped improve system reliability, data persistence, and overall performance, ensuring that the platform could efficiently handle multiple model executions and user interactions.



Luyanda Chitofu (Frontend & Testing): Pursuing a degree in Computer Information Systems with a Business Administration learning track. Luyanda focused on enhancing the user experience and implementing the automated generation of dynamic interfaces for the AI Model Repository Platform. Her primary contributions involved designing adaptable user interface components that dynamically adjust based on varying AI model inputs and outputs. This allowed the platform to support a wide range of AI models while maintaining a consistent and intuitive user experience. She worked closely with both frontend and backend logic to ensure that the dynamically generated

interfaces correctly displayed input fields, handled file and text submissions, and formatted outputs for users. Drawing from her academic coding experience in college, Luyanda applied skills developed through coursework in Python, Java, HTML, CSS, SQL, and JavaScript. These technical foundations supported her work on frontend development, UI logic, and integration with backend services. Her experience building web-based applications and working with databases helped ensure that user interactions were

properly handled and that dynamically generated components functioned reliably within the system architecture. Her academic interests include human-computer interaction and unit testing methodologies, which influenced her approach to validating interface behavior. She contributed to testing the dynamic UI across multiple input and output combinations to ensure that the platform adapted correctly to different model structures. Through these efforts, Luyanda helped improve usability, responsiveness, and consistency across the platform, ensuring that both developers and end users could interact with AI models efficiently.

Table of Contents

Biographical Background	4
SRS Documentation & Methodology.....	8
Core Functional Requirements	8
AI Integration & Monetization	9
Non-Functional & System Requirements	9
Summary of Implementation Status	9
System Construction & Methodology	9
Construction Technologies	10
Core Algorithm: Secure Model Execution	11
Design Diagrams	14
Evaluation and Key Results	18
Main Results	18
Experimental Results (Testing Table Summary)	18
Proof of Work (Screenshots)	18
Conclusion & Future Work.....	25
References.....	28
Git Repository.....	29

SRS Documentation & Methodology

The project followed a structured methodology aligned with the Software Development Life Cycle (SDLC).

Successful SRS Implementation:

- **User Authentication:** Secure registration and login with enforced validation rules.
- **Model Upload & Cataloging:** Automated storage of model files and metadata in a searchable catalog.
- **Dynamic Interface Generation:** The system provides specialized UI components based on whether a model requires text or file inputs.
- **Asynchronous Execution:** Django was configured to remain responsive while heavy AI tasks are handed off to the AI Suite.
- **Live Code Editing:** Service details can be edited after deployment of a service. This includes both model data and code files.
- **Pre-Publish Sandbox:** A feature to test code before final deployment was identified as a usability need so developers could test their code before it was accessible to all users on the platform.

Unfinished Items:

- **Payment System:** While the system has the basic framework of a credit-based monetization system, an actual payment system would still need to be implemented into the system before users could subscribe or buy more credits with more than just demonstration data.

Core Functional Requirements

These high-priority requirements define the essential operations of the platform:

- **Search and Discovery:** Users must be able to filter and search for AI models based on specific categories, frameworks, and tags.
- **Model Management:** The system must allow for the uploading of AI model files along with their metadata and maintain a complete version history for every uploaded model.
- **User Interactions:** Each model requires a dedicated page that hosts its documentation, files, version history, and community discussion threads.
- **Role-Based Access Control:**
 - Consumers can browse and download models.
 - Developers have additional permissions to upload and manage their own models.
 - Administrators have full system oversight, including the ability to manage users and toggle various platform features.

AI Integration & Monetization

These features distinguish the platform by automating documentation and managing resource usage:

- **Automated Documentation:** The system generates draft README files for new models and can summarize lengthy documentation into concise overviews.
- **Credit System:** To manage hosting and API costs, the system maintains a user "Credit Balance" and deducts credits for specific high-resource actions like model downloads.
- **Dynamic UI Generation:** A core technical success of the project is the ability to automatically provide specialized UI components based on whether a model requires text or file-based inputs/outputs.

Non-Functional & System Requirements

The following requirements ensure the platform remains secure, responsive, and maintainable:

- **Performance:** The system must handle concurrent user requests without degrading response times. Specifically, all AI model responses should be delivered within 10 seconds under normal conditions.
- **Security:** All user data must be encrypted to protect privacy during account management. Additionally, the system must enforce a zero-trust policy by isolating untrusted user code within secure sandboxes.
- **Availability:** Core repository functions like uploading and downloading must remain operational even if the auxiliary AI-enabled services become unavailable.

Summary of Implementation Status

According to the current project status, most of these core SRS requirements have been successfully implemented, including User Authentication, Model Cataloging, and Asynchronous Execution. One item currently identified as unfinished is the Pre-Publish Sandbox, which is scheduled as a secondary feature for future development.

System Construction & Methodology

The initial task of development was to facilitate communication between the three critical system components using Docker containers. After the components had been connected, the original development plan was to focus on the core functionalities uploading code, running code, receiving user input, and displaying output. Security was not a chief concern at this stage, as the team was trying to learn about the advanced API calls and Docker settings needed to get a working prototype. The first step of development was determining

the simplest database schema needed for the minimum viable project, followed by the construction of AI endpoints and a shared Docker volume structure to facilitate file transfers between the AI Suite and the system backend using metadata and file paths stored within the database. The entire process took several weeks, and the original code lacked proper security measures since user code was run directly in the AI Suite and had access to user information and critical system components.

Another chief concern at the design stage was determining how a user interface could be designed to accommodate a wide range of AI models. The design team eventually identified a key relationship between most stateless AI models; they accept an input and produce and output. Taking advantage of this fact, the team created an interface accepting text or file inputs and text or file outputs with support for eight text-to-filetype conversion or for custom files created in the user's code. The team later added support for multiple file outputs to accommodate models that produce more than one file. The flexibility of this system means that the system could be expanded in the future to automatically generate interfaces for state driven services such as chatbots.

After the demo was proven viable and met stakeholder expectations, focus shifted from quickly adding features to carefully redesigning components with security in mind. It was at this stage we transferred from a three-container system with all code running on the FastAPI to a four-container system that utilizes a temporary isolated sibling container of the AI Suite with a Python runtime environment used to execute untrusted code. A zero-trust policy was important for this application since it runs entrusted code in the system. This meant that Internet connection needed to be turned off, and the only communication the Sandbox could use would be JSON calls to and from the AI Suite. This was the longest part of development because in order to retain functionality for code that needs dependencies not already installed on the system, our team had to develop a provision that acted as a bridge between the Sandbox container, the internet, and the other system components. This, however, proved difficult as the code that had previously reached out to repositories on the internet had to be redirected to the dependencies that were being pre-installed in the AI Suite, stored on the platform, and accessed when the code runs. This was accomplished by developing a Regex parser algorithm to parse the code for patterns that commonly indicate that section of code is used to load model weights from the Internet. The solution may cause errors if code contains lesser known import statements for model weights or if the domain of said repository is not flagged as safe by our provisioner. Nonetheless, this solution was chosen for its robustness with a wide range of dependencies, while offering better security than allowing access to the entire internet.

Construction Technologies

The system utilizes a micro-services architecture built using Docker for scalability and security isolation.

- **Backend:** Django (Python) for core logic and the "single-point connection".

- **Database:** PostgreSQL for persistent record-keeping of model performance and user data.
- **AI Service:** FastAPI for high-performance API endpoints that communicate with Docker containers.
- **Isolation:** Docker SDK for Python to manage the lifecycle of temporary sandbox containers.

Core Algorithm: Secure Model Execution

The core logic involves a "Provisioner" that scans uploaded code for dependencies and a "Manager" that handles the sandbox lifecycle.

Algorithm Analysis: Model Asset Provisioner

The "Provisioner" used a Regular Expression (Regex) discovery algorithm to identify model loading patterns defined in the provisioner logic. In a worst-case scenario, the efficiency of the algorithm is defined as $O(N \times L \times M)$ where N is the number of files, L is the average lines per file, and M is the complexity of the regex pattern set. In most cases, the Regex parser caused no noticeable performance degradation compared to the code directly reaching out to the internet. This is the most fragile aspect of the code due to its reliance on regex expressions to detect model weights to download. It would need to be consistently maintained with regular updates as popular model weight repositories change to maintain functionality and security.

Algorithm Analysis: Entry-Point Discover

The logic to locate `main.py` in both the upload verification and execution route uses an Abstract Syntax Tree (AST) to traverse the code files to detect the `handle_request` signature. The theoretical complexity of this algorithm is $O(N \times L)$ where N is the number of files and L is the total number of lines across those files. The algorithm employs an early exit strategy that ends the process once the `handle_request` function is found, meaning that the worst possible case is when the algorithm cannot find the `handle_request` function. The impact is still minimal because the search completed in under a second in all tests, meaning it has no noticeable impact on user experience.

Pseudocode: Secure Execution Workflow

Python

Entry point for AI Suite execution

FUNCTION `execute_user_model(model_id, user_input)`:

 # 1. Fetch model metadata from Database and code from Shared

Volume

`path_to_code = fetch_from_database(model_id)`

`model_assets = fetch_from_shared_volume(path_to_code)` [cite: 39]

```

# Create a file with a hash value to detect changes to the code
since last execution
Meta_file = create_unique_hash(path_to_code)

# 2. Scan for dependencies using Regex Parser
dependencies = regex_scan(model_assets.main_file) [cite: 89, 95]

# 3. Initialize Sandbox with Resource Limits and network
restrictions
sandbox = docker.create_container(
    image="ai-runtime-base",
    mem_limit="2gb",          # Enforcement of resource limits
    network_disabled=True    # Zero-trust network isolation [cite:
49, 89]
)

# 4. Transfer code and execute
sandbox.mount(model_assets)
result = sandbox.run("python main.py", input=user_input) [cite:
40, 89]

# 5. Collect Metrics and Cleanup
log_metrics(execution_time, memory_usage, cpu_usage) [cite: 89]
sandbox.remove()

RETURN result [cite: 40]

```

Pseudocode: Model Upload Workflow

Python

Upload AI Service

FUNCTION upload_ai_service(upload_request):

1. Validation Phase

a. Ensure the user has developer or administrative privileges
if user.role == 'developer' or user.role == 'admin':

a. validate the model forum for required input

b. validate the model file to ensure a main.py and
 handle_request function is present

```
if uploaded_file.name.endswith('.zip'):
    is_valid, error_msg, hint =
validate_contract_in_zip(uploaded_file)
    elif uploaded_file.name.endswith('.py'):
        is_valid, error_msg, hint =
validate_contract_in_py(uploaded_file)
```

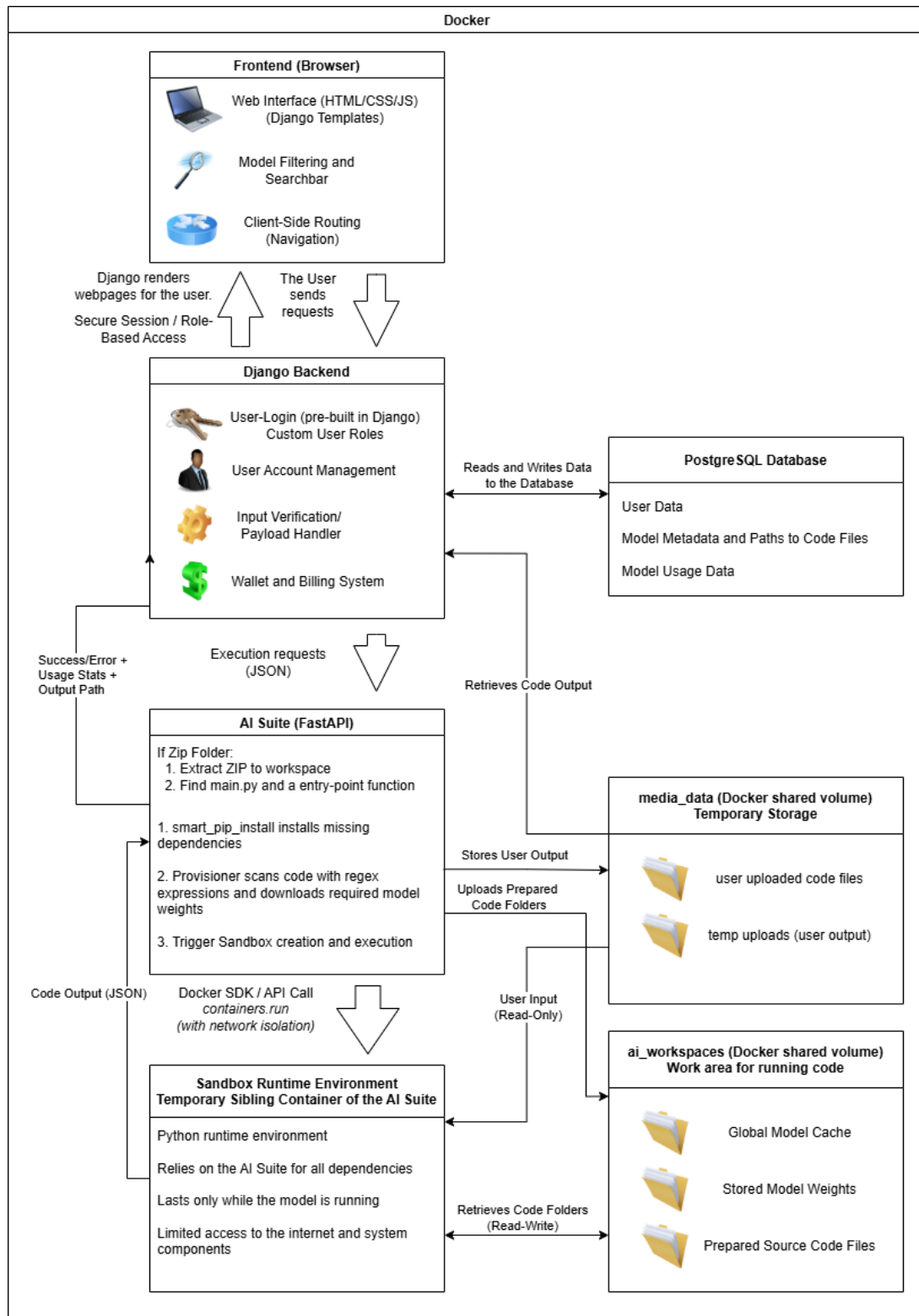
c. create a new entry in the AI_Model table in PostgreSQL

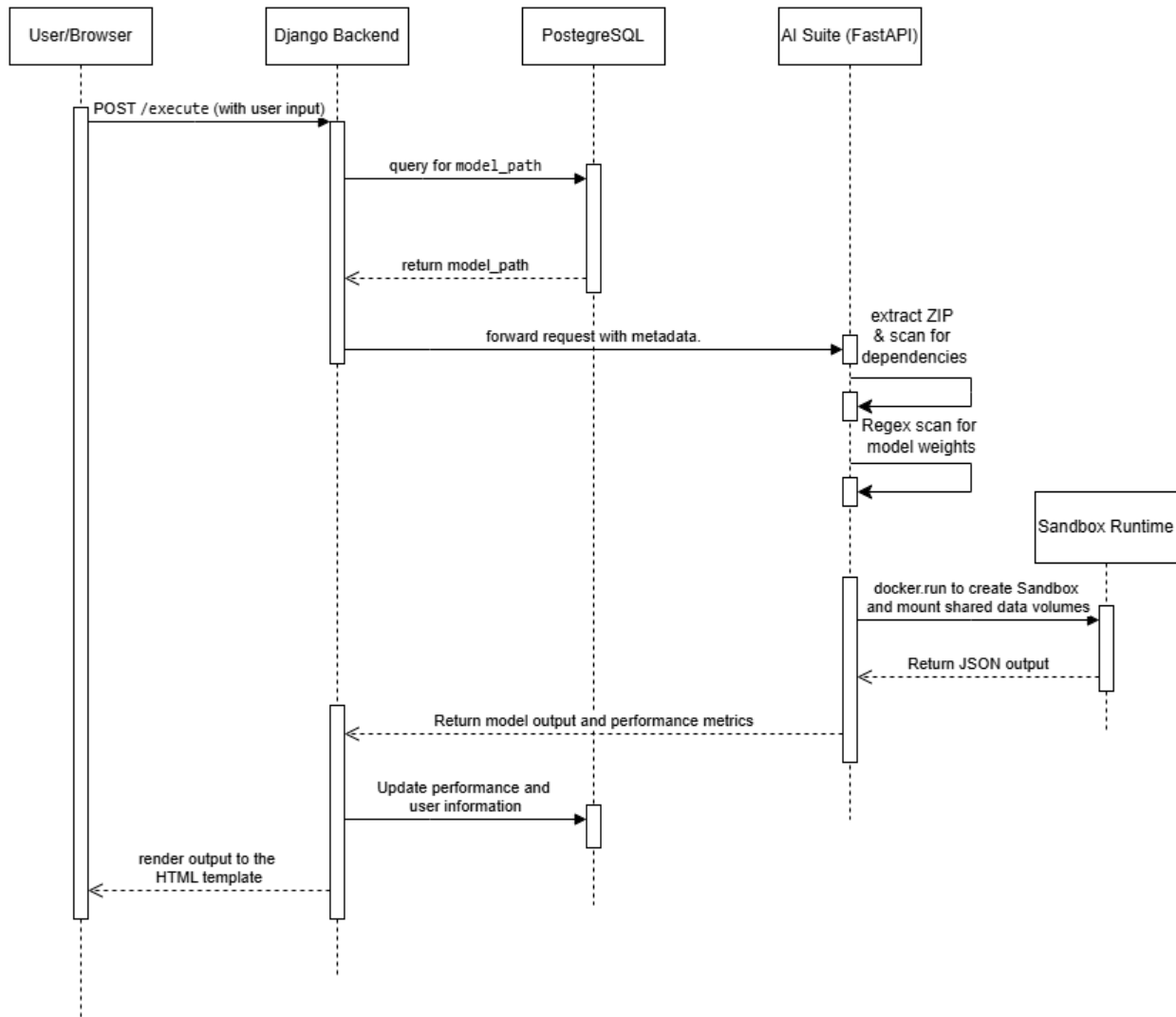
d. store the form metadata into PostgreSQL

e. save the code to the media shared volume
form.save

f. save the code into a shared volume
model_instance.save()

Design Diagrams

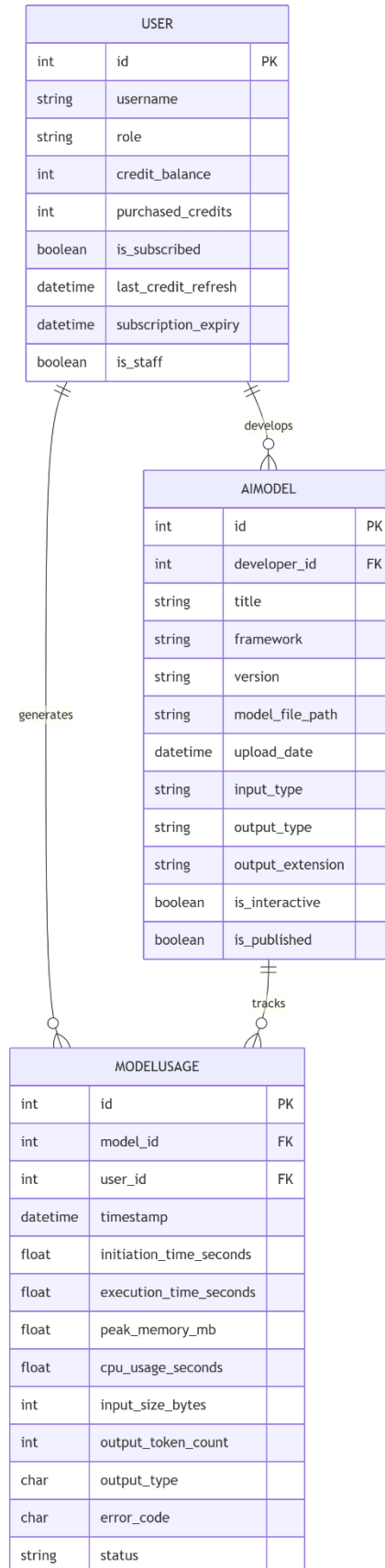




2

¹ Figure 1: Full-Stack Microservices Architecture. Illustrates the "single-point connection" where the Django backend coordinates data flow between the user interface, PostgreSQL database, and FastAPI execution services.

² Figure 2: System Logical Flow and Communication Sequence. Details the step-by-step communication process between the user's browser and the four main backend components during a model execution request.



The user table inherits Django’s AbstractUser, which handles industry-standard salted password hashing out of the box. For the ER diagram, we focused on the custom business logic fields, like role and credit_balance, while the authentication layer remains encapsulated within the framework’s secure identity management system. This ER diagram provides an overview of how the database is constructed as of April 16, 2026. The code defining this schema is defined in the models.py file of the project.

Feature	Description	Status
User Authentication	Secure registration, login, and role-based access control for consumers, developers, and administrators.	Implemented
Model Upload and Cataloging	Supports upload of Python and ZIP models with metadata, searchable tags, and version history.	Implemented
Dynamic Interface Generation	Automatically creates UI components based on model input and output requirements.	Implemented
Asynchronous Execution	Delegates AI processing to FastAPI services while maintaining Django responsiveness.	Implemented
Secure Sandbox Execution	Runs untrusted code in isolated Docker containers with resource and network restrictions.	Implemented
Dependency Provisioner	Parses code dependencies and provisions packages securely.	Implemented
Entry-Point Validation	Validates presence of required <code>main.py</code> and <code>handle_request()</code> function.	Implemented
Live Code Editing	Allows developers to modify code and metadata after deployment.	Implemented
Search and Discovery	Provides searchable catalogs for AI models and services.	Implemented
Execution Metrics Monitoring	Tracks CPU, memory, and performance statistics in PostgreSQL.	Implemented
Credit Monetization System	Prototype credit system for subscriptions and usage billing.	Partial
Pre-Publish Sandbox Testing	Developer testing environment before publishing services.	In Progress
Security Controls	Zero-trust protections including network isolation and resource limits.	Implemented
Microservices Architecture	Four-container architecture supporting modular scalability.	Implemented
Cloud Deployment Readiness	Planned support for production deployment and AWS hosting.	Planned

Table 1: Feature Description and Implementation Status

⁴ The "Implemented" status refers to features that have passed unit and integration testing within the local development environment. "Partial" and "In Progress" statuses indicate features currently undergoing active development in the alpha branch, while "Planned" features denote the final phase of the project lifecycle targeting AWS production parity

The table highlights a robust Microservices Architecture utilizing Django for frontend responsiveness and FastAPI for asynchronous AI processing. Key technical milestones achieved include Secure Sandbox Execution via Docker and an automated Dynamic Interface Generation system. The current development status indicates that while core security and execution features are fully implemented, the project is currently finalizing developer testing environments and transitioning toward cloud-native deployment.

Evaluation and Key Results

Main Results

- **Functional Success:** The system correctly identifies entry points (`handle_request`) within uploaded .zip folders and executes them.
- **Integration:** Successful communication was verified between the Django hub and FastAPI status endpoints.
- **Metric Tracking:** Peak memory, CPU usage, and execution status are consistently saved to the PostgreSQL database.

Experimental Results (Testing Table Summary)

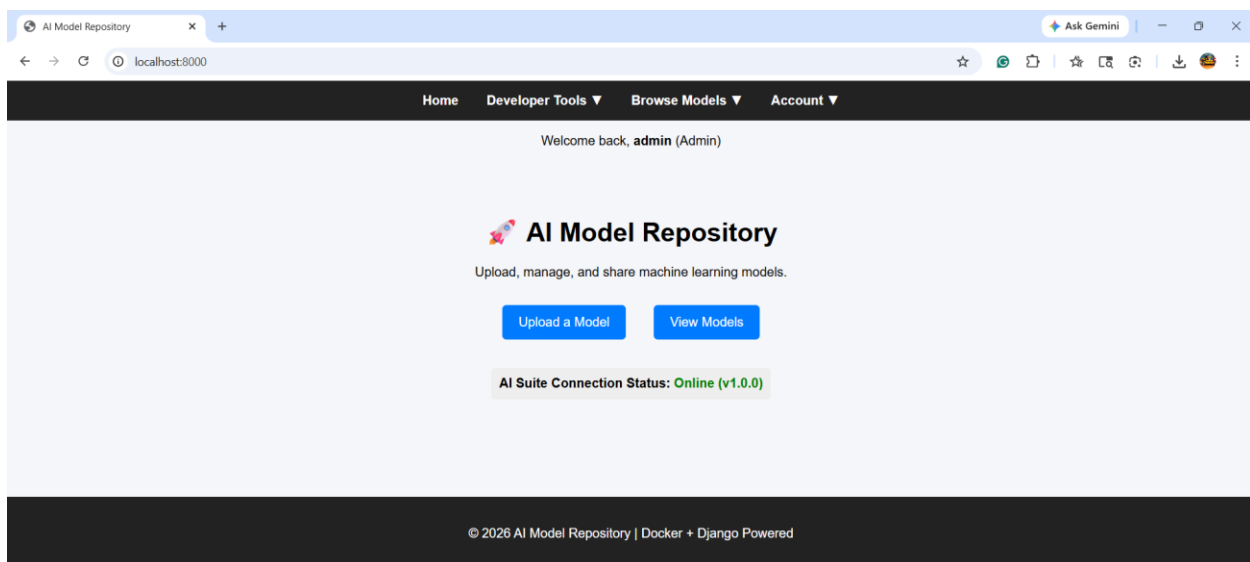
Category	Test Case	Status	Notes
Unit	Regex Dependency Parser	 Pass	Identified torch/transformers correctly.
Security	Network Isolation	 Pass	Blocked outside website access from sandbox.
Security	Path Traversal	 Failed	Sandbox could view the shared /app/media/ directory.
Stress	Concurrent Execution	 Pass	3 models run simultaneously with <10s delay.

Proof of Work (Screenshots)

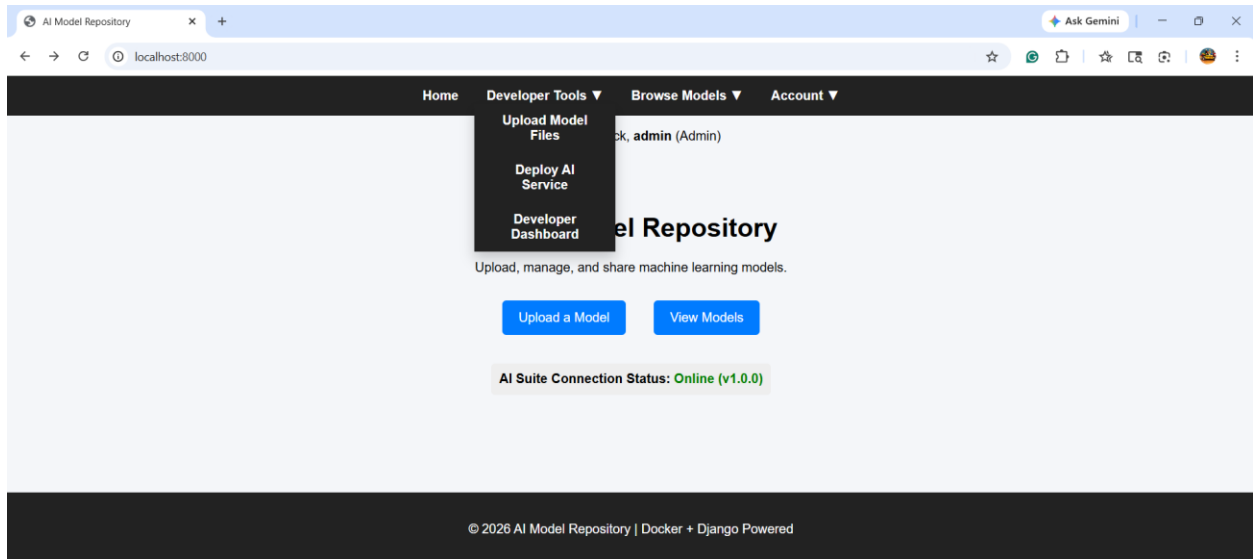
Screenshots in the critical report confirm that the system provides specific error messages for missing entry points in .zip and .py files. Terminal outputs confirm the system correctly triggers OOM (Out of Memory) kills with exit code 137 when usage limits are exceeded by developer models. Screenshots found in the critical testing report also detail the performance details of the system in conditions with more than one concurrent user.

The screenshot shows a web browser window with the address bar displaying 'localhost:8000/signup/'. The page has a dark navigation bar with links for 'Home', 'Upload Model', 'Browse Models', and 'Account'. The main content area features a 'Join the Repository' form. The form includes a 'Username' input field with a note: 'Required, 150 characters or fewer. Letters, digits and @/./+/_ only.' Below this is a 'Role' dropdown menu with 'Consumer' selected and 'Developer' as an option. A list of password requirements is shown: 'Your password can't be too similar to your other personal information.', 'Your password must contain at least 8 characters.', 'Your password can't be a commonly used password.', and 'Your password can't be entirely numeric.' There is a 'Password confirmation' input field with the instruction 'Enter the same password as before, for verification.' A blue 'Create Account' button is at the bottom of the form, followed by a link: 'Already have an account? [Login here](#)'.

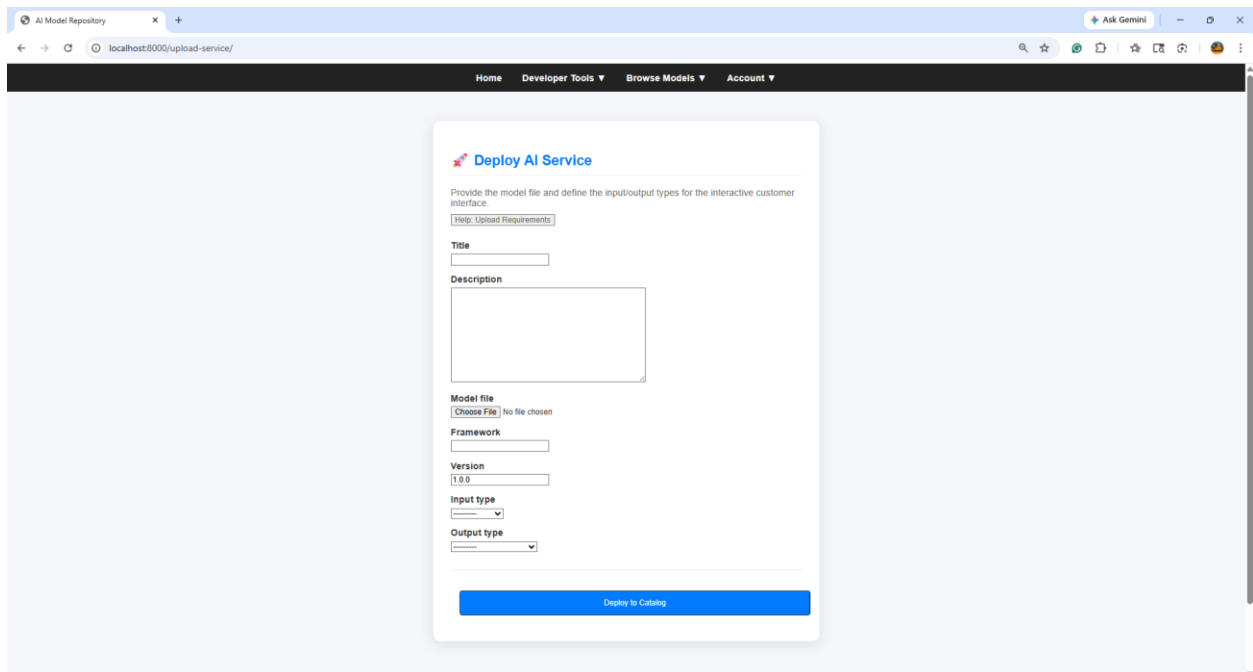
The signup form allows users to register as consumers or developers depending on their uses for the platform. For security, administrative accounts can only be created by directly entering a create superuser command in the backend terminal.



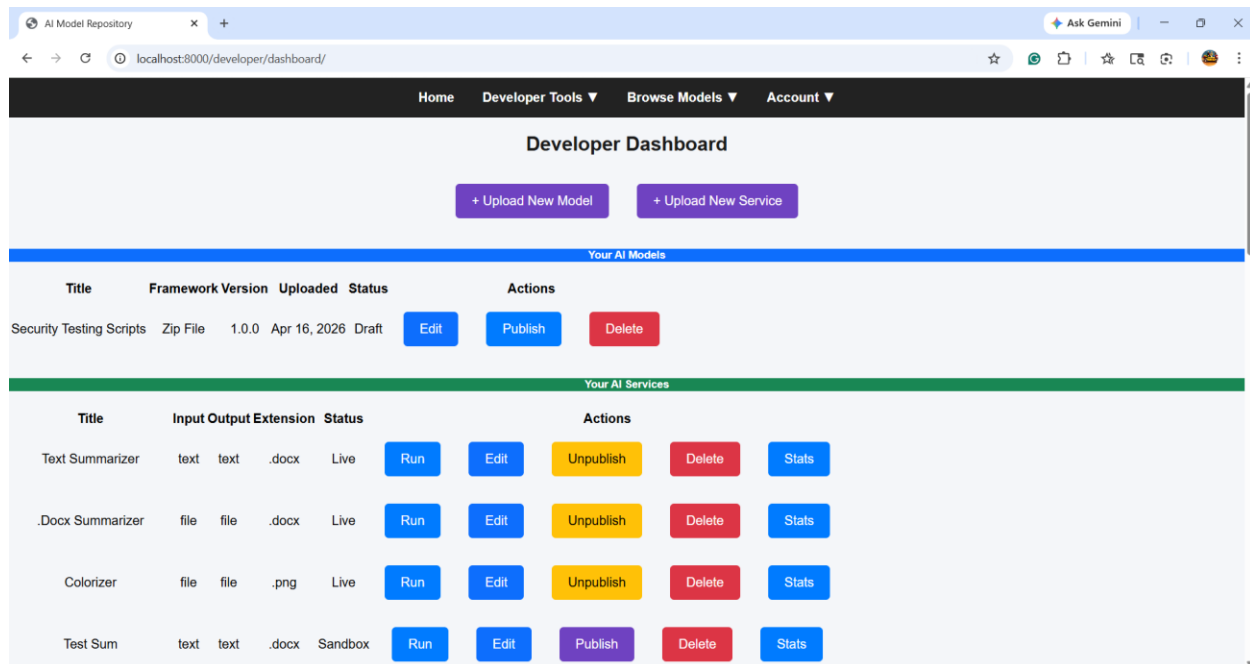
A view of the home screen of the application. The presence of the developer tools in the navbar indicate that this user has developer or administrator privileges. The AI Suite Connection Status also indicates that the connection between the Django backend and the FastAPI AI Suite is successful as it makes an asynchronous API call to the AI Suite to verify this connectivity.



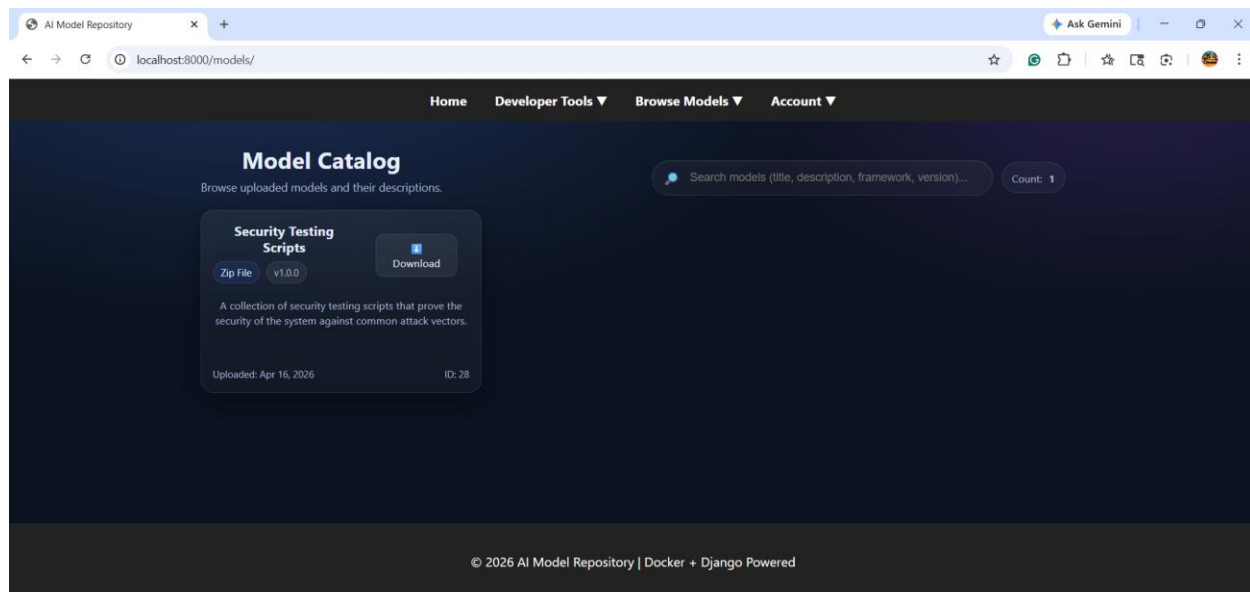
Developers have the option to upload model code so others can view the code, deploy an AI service to run on the system, or view statistics and manage existing services at the developer dashboard.



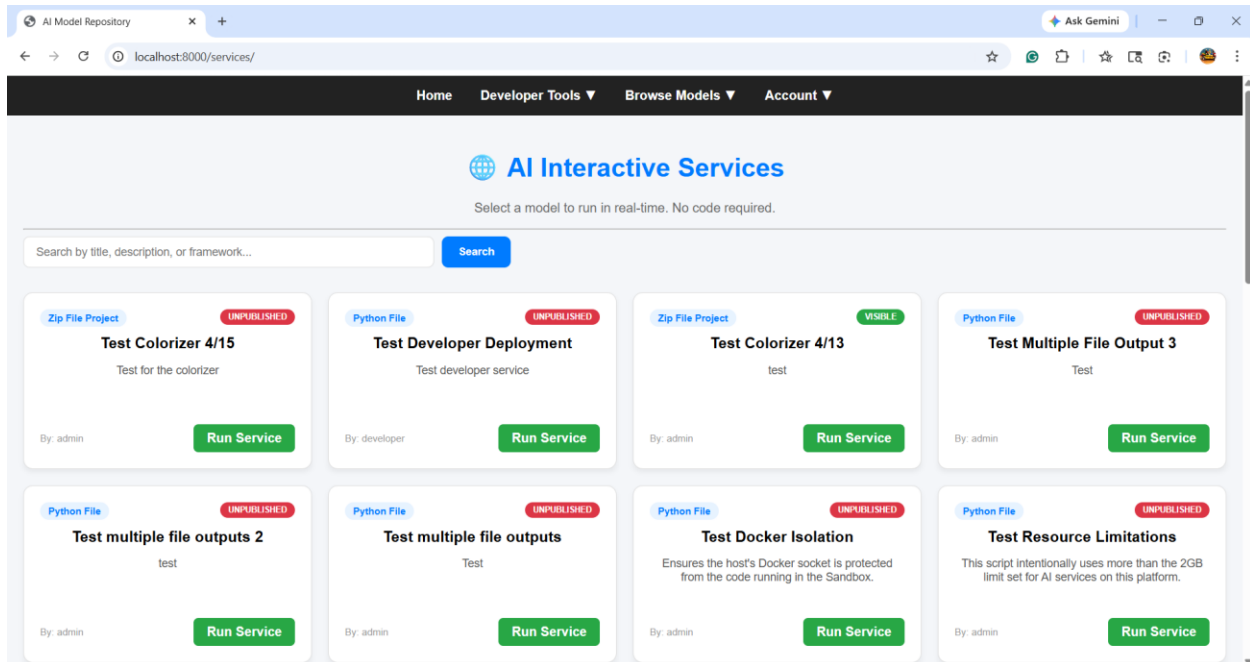
The form to deploy AI services is what allows developers to upload information about their model, upload the model code, and define the input and output types. Input and output types are what gives the system the necessary information to construct an interface for the code. The form to upload model code is similar but doesn't include the input and output type fields needed for the interface. The system can be configured to automatically convert text output to seven different file types, a custom file can be returned from the code, or text can be sent directly to the interface.



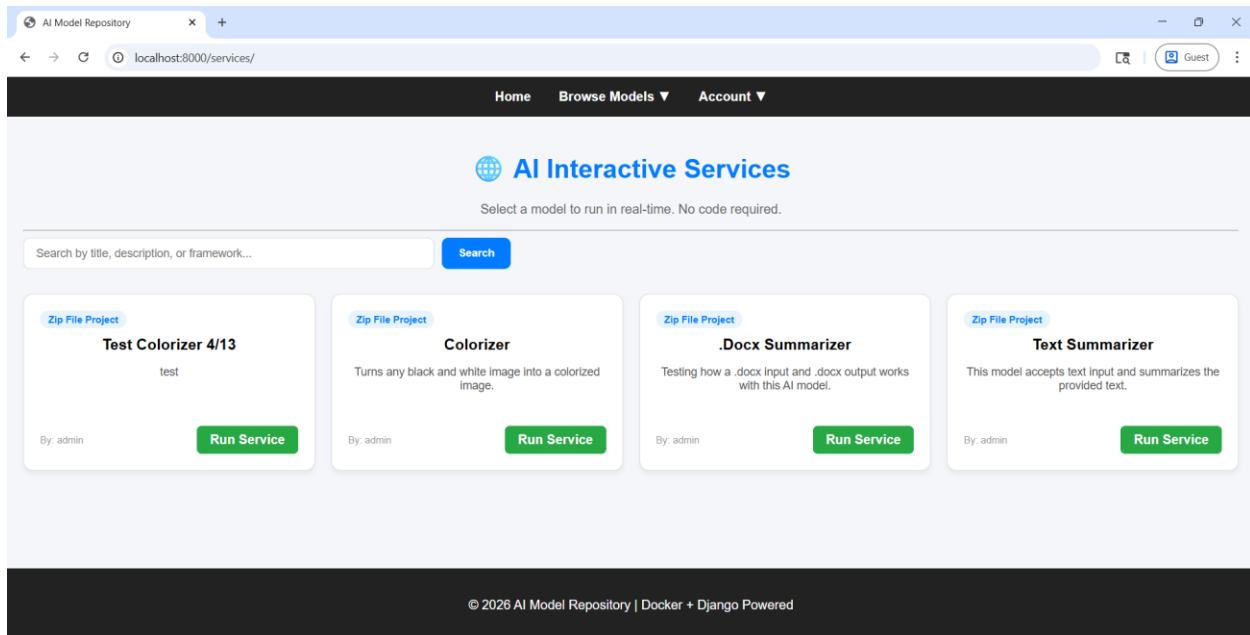
The developer dashboard lets developers test, edit, publish, delete, and analyze their AI services.



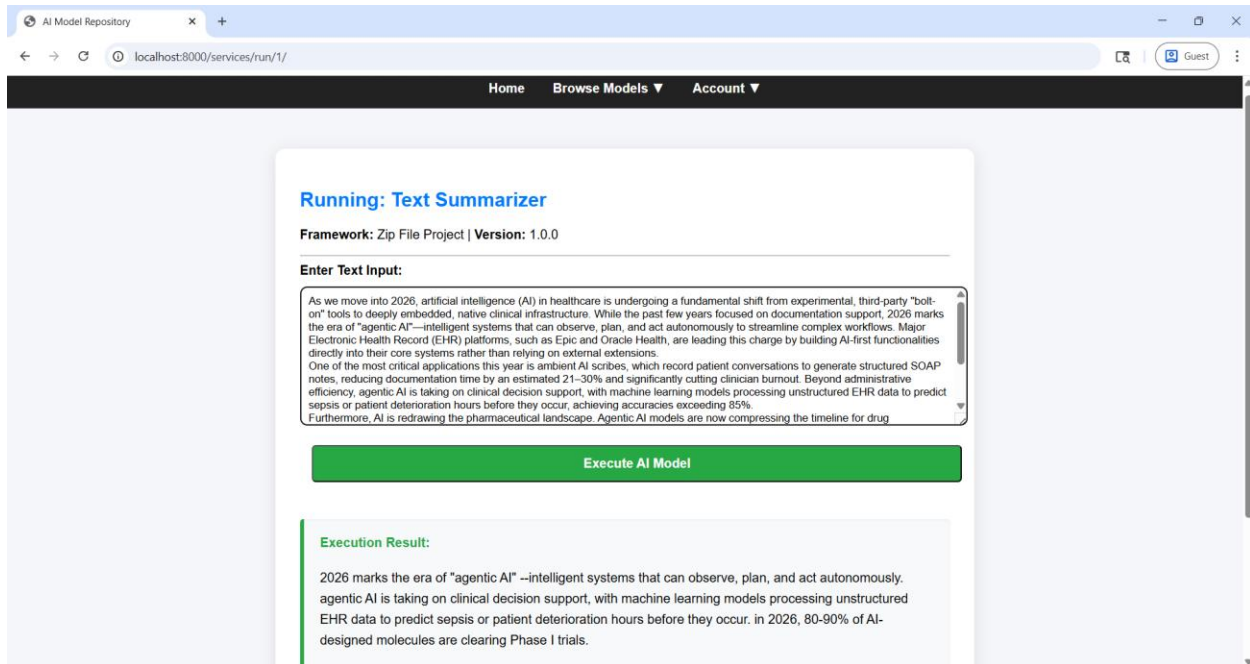
The model catalog is available for users to download publicly available code. Similar to GitHub, this feature is used for developers that want to offer their code for download instead of just making it available as a service. The catalog also lets users search for specific models based on title, framework, description, or username. The styling of this page was slightly inconsistent as it got less development time compared to the service deployment system.



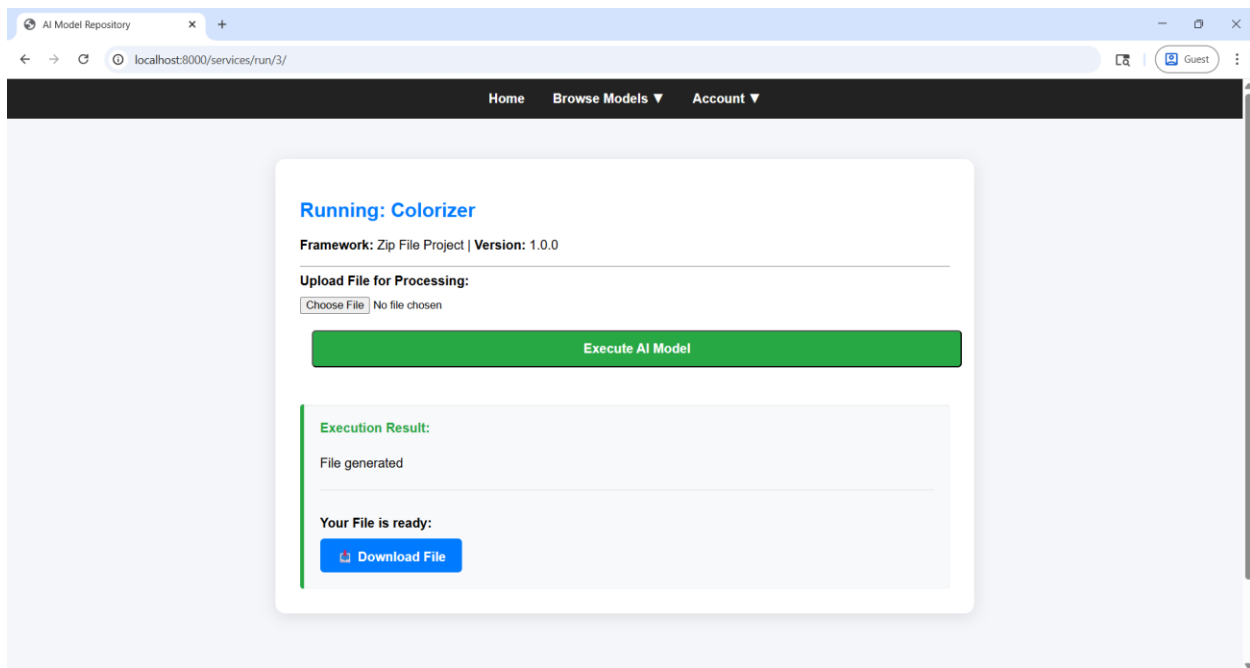
The AI Service catalog is available to all visitors of the site and allows users to browse available services. The search bar functions the same as the model catalog. Clicking run service loads the interface for the service. This developer uploaded many models for testing. Models marked unpublished cannot be seen by other users, while models marked visible can be seen. Only the user who created the service can see an unpublished service in the catalog.



The same catalog as seen by a consumer. Only the models that have been published and marked visible in the developers' view are visible to consumers and other developers.



Standard interface for a text input/text output service



Standard interface for a file input/file output service



Example input vs. output of the colorizer (random photo colorizer model from GitHub)

AI Model Repository

localhost:8000/account/

Home Browse Models Account

Account Management

Manage your credits, subscriptions, and usage.

Available Credits

5

Credits reset to 5 daily.
Cost: 10¢ per credit
Credits to purchase:

[Proceed to Checkout](#)

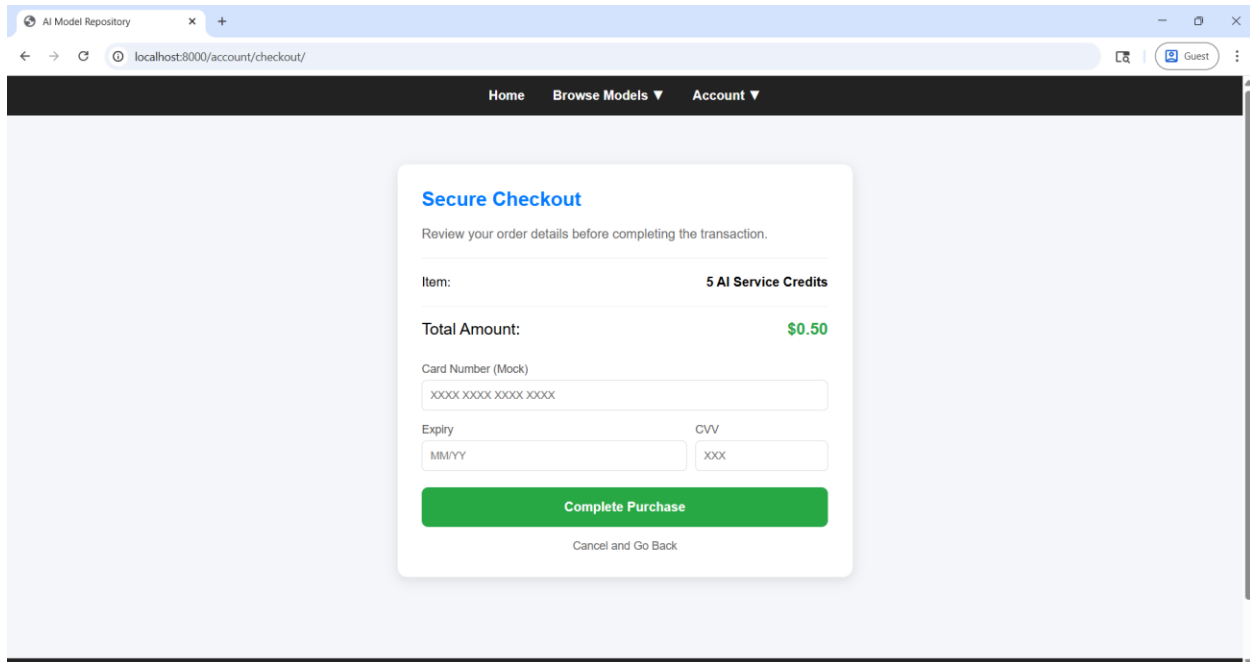
Premium Access

Get unlimited model executions.

\$11.99 / month

[Subscribe Now](#)

© 2026 AI Model Repository | Docker + Django Powered



The account management and checkout page lets consumers manage subscriptions and purchase more usage credits. Each time a model is run successfully a credit is deducted from an unsubscribed user's account. The checkout is only a demo and does not accept sensitive information currently.

Conclusion & Future Work

Summary

The AI Model Repository Suite successfully addressed the "fragmentation gap" in modern AI development by providing a unified, microservices platform for secure model hosting. The project achieved its primary goal: allowing developers to deploy Python-based AI services without requiring manual frontend development or complex cloud orchestration knowledge. Through the implementation of a "single-point connection" architecture using Django, PostgreSQL, and FastAPI, the team demonstrated that a modular system can effectively manage the complexities of dynamic dependency resolution and automated interface generation.

Lessons Learned

- **Mapping College Coursework:**
- The successful construction of the platform relied on a synthesis of diverse learning tracks within the Computer Information Systems program. Cybersecurity coursework, including **CY 201 (Introduction to Cybersecurity)** and **CY 450 (Intro to Cyber Operations)**, provided the foundation for the system's zero-trust execution

policy and network isolation. Development and system management skills were grounded in core classes such as **CS 440 (Database)**, **IS 340 (Information Technology)**, and **IS 448 (IS/IT Project Management)**, which were critical for designing the Django-PostgreSQL architecture and managing complex data contracts between microservices. **CS506 (Distributed Cloud Computing)** provided the foundation for building a cloud infrastructure compatible application with Django. AI model structure, training, deployment, and code details were taught in **CS591(Advanced Artificial Intelligence)**, and proved critical when developing a platform that required a deep understanding of machine learning code. **CS445 (Software Engineer I)** prepared our team to develop a large-scale project while managing stakeholder needs within limited time constraints. Additionally, the user interface and platform logic were informed by **IS 420 (Human Computer Interaction)** and **IS 440 (Web Design for Electronic Commerce)**, ensuring the final product met both technical and usability standards. This project demonstrated that while specialized knowledge is essential, a full-scale application requires the integration of skills across development, security, and systems analysis.

- **Technical:** The implementation of the secure sandbox architecture highlighted the challenge of balancing functionality with security. Code that operates correctly in an open environment often fails when subjected to the strict resource and network constraints of an isolated container.
- **Security:** Testing proved that security is an ongoing process rather than a static feature. Formal vulnerability assessments were significantly more effective than initial design hypotheses in identifying critical flaws, such as the path traversal vulnerability found in the shared media directory.
- **Soft Skills:** Managing a distributed microservices project required rigorous synchronization between the "hub" (Django) and "service" (FastAPI) developers to maintain a consistent and reliable data contract.

Future Work

- **Enhanced Isolation:** Implement stricter volume mounting to prevent sandbox access to the shared media folder. A unique UUID is currently used to provide output to the correct users, but a unique volume for each service would further isolate the files and make data exfiltration more difficult.
- **Usability:** Improve front-end error reporting so developers understand why their code hit a resource limit or failed execution. Although we currently have robust error checking to find errors, they are typically unique to the Docker containers and may not correctly transfer to the template in the backend. This is a Docker related issue and would require extensive and intentional error testing to identify, meaning that it was better left to future development as the developer is able to navigate and correct most errors successfully based on what is returned in the output of the browser template.
- **Deployment:** Refine the system for full-scale production deployment based on the identified roadmap. While the server runs well on local systems it would be a critical

step to test its deployment on multiple operating systems and even cloud-based hosting platforms such as AWS.

- **Monetization Improvements:** There is currently a DEMO credit system that requires consumers to purchase credits or subscribe to the platform once they reach their daily usage limit. While this provides the framework, we have yet to implement actual credit card processing systems or ways for the user to pay. Integration with payment systems such as PayPal would also be critical since it would provide more peace of mind to some users when subscribing to the system or buying more credits.

References

“AWS Lambda.” *AWS*, 18 Feb. 2026, <https://aws.amazon.com/lambda/>.

Based on the provided code how do I fix the issue that is preventing the backend from accessing the correct folder in the shared Docker volume” prompt. *Copilot*, “Smart” version, Microsoft, 10 Mar. 2026, <https://copilot.microsoft.com/>.

“Building Web App with Django and FastAPI.” *GeeksforGeeks*, 28 Jun. 2024, <https://www.geeksforgeeks.org/python/building-web-app-with-django-and-fastapi/>

Charboneau, Tyler. “Top Tips and Use Cases for Managing Your Volumes.” *Docker*, 22 June 2022, <https://www.docker.com/blog/top-tips-and-use-cases-for-managing-your-volumes/>.

“Creating a secure sandbox architecture in Docker with pre-loaded dependencies for AI models using FastAPI” prompt. *Gemini*, 3.0 version, Google, 27 Mar. 2026, <https://gemini.google.com/app>.

“FastAPI Documentation: Asynchronous Web Frameworks.” *FastAPI*, 16 Feb. 2026, <https://fastapi.tiangolo.com/#license>.

Haig, Lance. “How to Dockerize a Django App: Step-by-Step Guide for Beginners.” *Docker*, 8 Jan. 2025, <https://www.docker.com/blog/how-to-dockerize-django-app/>.

Hash Block. “Which Database is Best for Your Project? A Complete Guide for Real-World Use Cases.” Medium, 21 Jul. 2025, <https://medium.com/@connect.hashblock/which-database-is-best-for-your-project-a-complete-guide-for-real-world-use-cases-e37d3848f9d2>.

“How to build a platform for AI models where a developer uploads their code and the system generates the interface.” prompt. *Gemini*, 3.0 version, Google, 16 Feb. 2026, <https://gemini.google.com/app>.

Qian, Mian, et al. “Survey of Artificial Intelligence Model Marketplace.” *Future Internet*, vol. 17, no. 1, Jan. 2025, p. 35. *EBSCOhost*, <https://doi.org/10.3390/fi17010035>.

Shubham. “Model Wights File Formats in Machine Learning.” *LearnOpenCV*, 12 May 2025, <https://learnopencv.com/model-weights-file-formats-in-machine-learning/>.

“Testing in Django.” *Django Documentation*, 31 Mar. 2026, <https://docs.djangoproject.com/en/6.0/topics/testing/>.

The Capstone Crew. "Critical Testing Report." 8 Apr. 2026. Capstone Experience CS 499, student report.

Git Repository: <https://github.com/KySchwartz/ai-model-repository>